EX03 - Solver cuadratico - Condicionales anidados y numeros complejos

Guia paso a paso para principiantes

Esta guia sigue el archivo real `ex03_quadratic_solver.py`. Se explica primero el problema y despues el codigo desde el punto de vista de alguien que esta empezando Python.

1. Que queremos resolver

Resolver a*x**2+b*x+c=0 contemplando cuando deja de ser cuadratica, cuando hay una raiz doble, dos reales o dos complejas.

Variables / elementos del problema

Nombre	Significado
a,b,c	coeficientes de la ecuacion
D	discriminante b**2-4*a*c
x	raiz o raices

Ecuacion o logica en papel

```
a*x**2 + b*x + c = 0
D = b**2 - 4*a*c
x = (-b +/- sqrt(D))/(2*a)
```

Comprobacion manual

Para a=1,b=-3,c=2: D=1 y las raices son 2 y 1. Para el caso activo a=0,b=2,c=1, la ecuacion pasa a ser lineal: 2*x+1=0 -> x=-0.5.

2. Lo que ya sabemos y lo nuevo

Ya visto:

- variables
- if / elif / else
- np.sqrt()

Nuevo en este ejercicio:

- import numbers
- isinstance()
- if anidados
- complex()
- conjugate()
- comprobacion sustituyendo la raiz

3. Conceptos Python nuevos

1. Validar tipo

Sintaxis: `isinstance(a, numbers.Real)`

isinstance(objeto,tipo) devuelve True si el objeto pertenece al tipo indicado. numbers.Real admite numeros reales como int y float.

2. if anidado

Sintaxis: `if a == 0: if b == 0: ...`

Un if dentro de otro solo se evalua si antes se ha entrado en el bloque exterior.

3. Numero complejo

Sintaxis: `complex(real, imag)`

Python puede representar un numero con parte real e imaginaria. complex(0,2) representa 0+2j.

4. Metodo

Sintaxis: `x1.conjugate()`

El punto accede a una operacion asociada al objeto x1. conjugate() devuelve el conjugado del complejo.

4. Codigo paso a paso

Paso 1. Validar a,b,c

```
if isinstance(a, numbers.Real) == False:
    print(...)
```

El ejercicio intenta asegurar que cada coeficiente sea real antes de resolver.

Paso 2. Tratar a=0 primero

```
if a == 0:
    ...
```

Si a=0 desaparece x**2 y la formula cuadratica no puede usarse porque dividiria por 2*a.

Paso 3. Calcular D en el caso general

```
discriminant = b**2 - 4 * a * c
```

D es una variable derivada que decide que tipo de raiz tendremos.

Paso 4. Elegir segun D

```
if discriminant < 0:
    ...
elif discriminant == 0:
    ...
else:
    ...
```

D<0 produce complejas; D=0 una raiz doble; D>0 dos reales.

Paso 5. Comprobar sustituyendo

```
check_1 = a * x1**2 + b * x1 + c
```

Una raiz correcta debe hacer que la ecuacion sea 0 o muy cercana a 0 numericamente.

5. Flujo de ejecucion

```
a,b,c
   -> type validation
   -> special cases (a=0, b=0, c=0)
   -> discriminant
   -> roots
   -> substitute back
```

6. Resultado / comprobacion esperada

Con el caso activo a=0,b=2,c=1, el archivo devuelve x=-0.5 y la comprobacion b*x+c=0.0.

7. Errores tipicos al empezar

- Usar la formula cuadratica con a=0.
- Confundir = con ==.
- Perder la sangria de un if anidado.
- No validar las raices al final.

8. Si solo recuerdas cinco cosas

- == compara; = asigna.
- isinstance() valida tipos.
- Los casos especiales van antes del caso general.
- D decide el tipo de raiz.
- Sustituir la solucion es una validacion independiente.

9. Mini practica

Cambia un unico input. Antes de ejecutar, escribe en un comentario que esperas que cambie y por que. Despues ejecuta y compara la prediccion con la salida.

Criterio de cierre

Puedes continuar cuando puedas explicar el problema, la ecuacion/logica, los conceptos Python nuevos y el flujo de ejecucion sin limitarte a leer el codigo.